

Trabajo Práctico N° 2
Programación Orientada a Objetos

Batalla de Magos contra Mortífagos

Modelado y simulación de un sistema de combate por turnos

Integrantes

Iascas, Facundo David - D.N.I. 45.321.120

Bonfigli, Leonardo - D.N.I. 44.159.042

Caccavari, Franco - D.N.I. 45.128.064

Casale Benavente, Pedro Santino - D.N.I. 46.701.634

Roig Moliterno, Iván Nicolás - D.N.I. 45.689.895

Comisión 02-2900

Docentes: Gasior, Federico Mauro · Lanzilotta, Hernán Gabriel

1.er Cuatrimestre 2026

Junio de 2026

Índice

1. Introducción	2
2. Análisis del problema	3
3. Diseño de la solución.....	3
3.1. Organización en paquetes.....	3
3.2. Diagrama de clases.....	4
4. Jerarquía de personajes: herencia y polimorfismo	4
5. Hechizos: una interfaz, muchos comportamientos	5
6. Creación centralizada: el patrón Factory	7
7. Gestión de batallones y colecciones	7
8. Reglas de la batalla	8
9. Bonus extra 1: Hechizos con efectos de estado temporales	8
10. Bonus extra 2: Comandante reacciona a muerte de aliado.....	9
11. Pruebas unitarias con JUnit.....	10
12. Decisiones de diseño y dificultades	11
13. Conclusiones.....	11
14. Referencias	12

1. Introducción

El informe documenta el desarrollo del segundo trabajo práctico de la materia Paradigmas de Programación, cuya consigna nos pidió resolver un problema concreto aplicando de punta a punta el paradigma orientado a objetos. La temática elegida por la cátedra fue una batalla entre magos y mortífagos: dos bandos enfrentados, cada uno con sus propios personajes, hechizos y habilidades, que combaten por turnos hasta que uno de los dos queda en pie.

El proyecto se desarrolló en Java (JDK 17) sobre Eclipse, y las pruebas unitarias se escribieron con JUnit 5. El código quedó organizado en cuatro paquetes (personajes, hechizos, Combate y Test) y la aplicación se puede ejecutar de dos formas: corriendo los tests o ejecutando un main que arma los dos batallones y los hace pelear ronda a ronda, mostrando todo por consola. A lo largo del informe vamos a ir explicando las decisiones de diseño que tomamos y el porqué de cada una.

2. Análisis del problema

Antes de escribir código separamos lo que el sistema tenía que hacer de cómo lo íbamos a resolver. Del enunciado salieron cuatro grandes piezas:

- **Personajes.** Hay dos facciones (magos y mortífagos) y dentro de cada una conviven varios tipos (aurores, estudiantes, profesores; seguidores y comandantes). Todos comparten una base común (nombre, nivel de magia, vida, hechizos que conocen) pero se diferencian en cómo pelean.
- **Hechizos.** Cada conjuro hace algo distinto: unos dañan, otros curan, otros dejan un estado en el objetivo. Tenía que ser posible sumar un hechizo nuevo sin tocar los que ya estaban.
- **Batallones.** Los personajes se agrupan en equipos. El batallón gestiona quién está en pie, a quién se ataca y lleva un registro de lo que va pasando.
- **La batalla.** Se juega por rondas y turnos, con reglas claras: quien tiene la vida en cero queda fuera, un batallón no puede repetir un hechizo dentro del mismo turno, y la cosa termina cuando un bando se queda sin nadie.

Un punto que nos ayudó a encaminar el diseño es que la diferencia entre tipos de personaje tenía que surgir del polimorfismo y no de preguntar el tipo con condicionales. Esto nos obligó a pensar desde el principio en una jerarquía de clases bien armada y en métodos que cada subclase redefine a su manera.

3. Diseño de la solución

3.1. Organización en paquetes

Dividimos el proyecto en paquetes según la responsabilidad de cada grupo de clases. La idea fue que al abrir el proyecto cualquiera entienda de qué se trata cada cosa sin tener que leer todo:

Paquete	Responsabilidad	Clases principales
---------	-----------------	--------------------

personajes	Jerarquía de combatientes, sus estados y su fábrica.	Personaje, Mago, Mortifago, Auror, Estudiante, Profesor, Seguidor, Comandante, Estado, PersonajeFactory
hechizos	La interfaz Hechizo, sus 14 implementaciones y la fábrica de hechizos.	Hechizo, TipoMagia, HechizoFactory + los 14 conjuros
Combate	La lógica de los equipos y el programa principal.	Batallon, BatallaMagosVsMortifagos
Test	Las pruebas unitarias con JUnit 5.	PersonajesTest, hechizosTest, CombateTest

Tabla 1. Estructura de paquetes del proyecto.

3.2. Diagrama de clases

La siguiente figura resume la arquitectura: la jerarquía de personajes, la interfaz de hechizos con sus implementaciones y las fábricas que las conectan con los batallones.

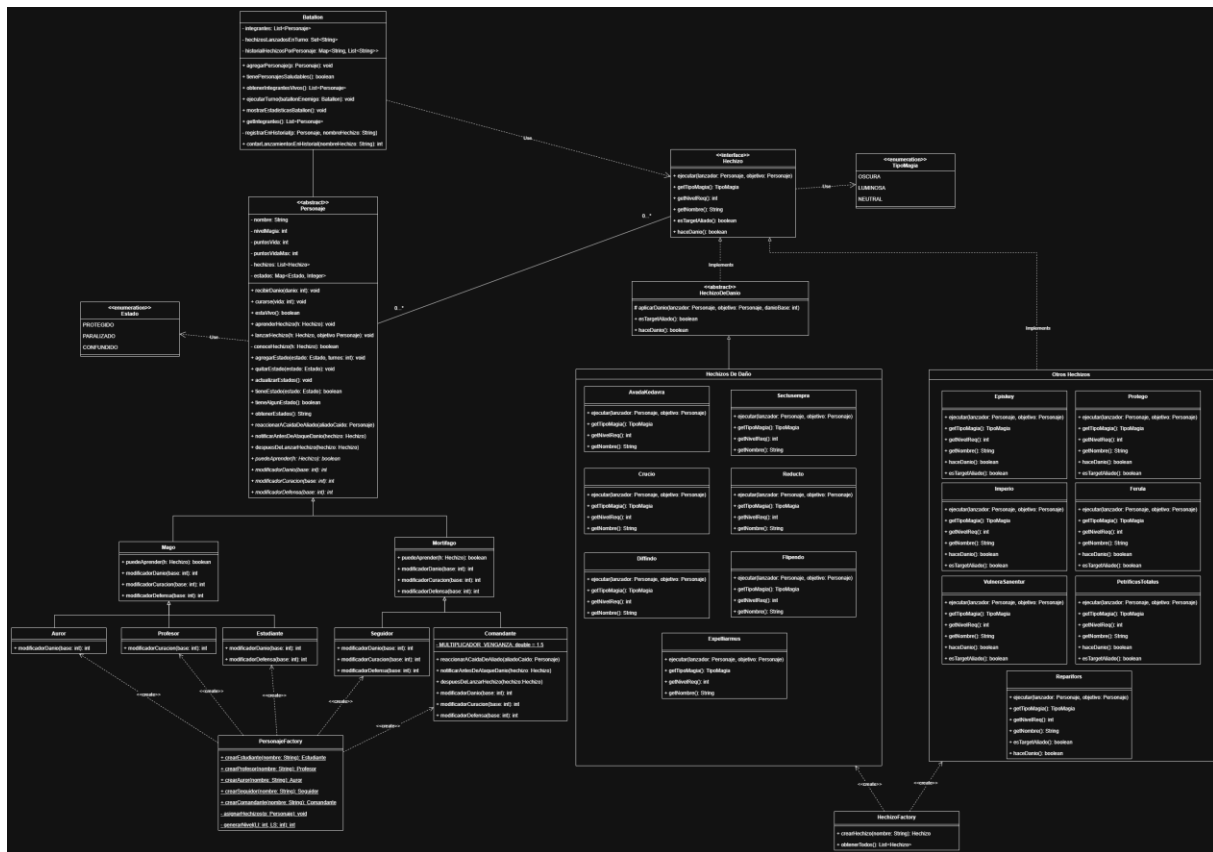


Diagrama de clases del sistema.

4. Jerarquía de personajes: herencia y polimorfismo

El corazón del diseño es la clase abstracta **Personaje**. Ahí viven los atributos y comportamientos que todos comparten: el nombre, el nivel de magia, los puntos de vida (y un tope máximo, para que nadie se cure hasta el infinito), la lista de hechizos que conoce y el conjunto de estados que tiene encima. También resuelve la mecánica común: recibir daño, curarse, aprender un hechizo o lanzarlo.

Hay tres cosas que dejamos como métodos abstractos, porque la respuesta depende de qué clase de personaje sea:

```
public abstract boolean puedeAprender(Hechizo h);
public abstract int modificadorDanio(int base);
public abstract int modificadorCuracion(int base);
public abstract int modificadorDefensa(int base);
```

Fragmento 1. Métodos abstractos de Personaje que cada subclase resuelve a su manera.

De la misma forma un hechizo de daño nunca pregunta quién lo lanza, simplemente le pasa su daño base al lanzador y deja que él lo ajuste. Así, el mismo conjuro pega distinto según quién lo tire, sin usar `if` ni un `instanceof` de por medio. Por ejemplo, así está escrito `Expelliarmus`:

```
@Override
public void ejecutar(Personaje lanzador, Personaje objetivo) {
    aplicarDanio(lanzador, objetivo, 20);
}
```

Fragmento 2. El hechizo no conoce el tipo del lanzador; solo le pide que modifique el daño base.

La jerarquía quedó en dos niveles. Primero Mago y Mortífago, que fijan el "carácter" de cada bando: el mago cura mejor y se defiende un poco más, mientras que el mortífago pega más fuerte pero cura peor. Después, las subclases concretas (Auror, Estudiante, Profesor, Seguidor y Comandante) afinan esos valores llamando a `super` y sumando o restando su propio diferencial. Un Auror, por ejemplo, hereda el daño de Mago y le agrega +25. El resultado neto se ve en la siguiente tabla:

Clase	Modif. daño	Modif. curación	Modif. defensa
Mago (base)	+0	+10	+5
Auror	+25	+10	+5
Estudiante	-5	+10	+15
Profesor	+0	+25	+5
Mortífago (base)	+10	-5	+0
Seguidor	+25	-25	-10
Comandante	+25	-15	-5

Tabla 2. Modificadores netos sobre el valor base, obtenidos por la cadena de herencia.

Esta tabla refleja bastante bien la idea: los aurores son los pegadores de elite del lado luminoso, los estudiantes son flojos atacando pero aguantan, el profesor es el sanador del grupo, y del lado oscuro tanto seguidores como comandantes son letales pero pésimos curándose. La gracia es que toda esa diversidad se logra redefiniendo unos pocos métodos, no escribiendo lógica especial para cada uno.

5. Hechizos: una interfaz, muchos comportamientos

Para los hechizos descartamos la idea de una clase gigante con un `switch` por nombre de conjuro: eso habría sido justo lo que la consigna pedía evitar. En su lugar definimos una interfaz `Hechizo` y cada

conjuro es una clase aparte que la implementa. Esto es el patrón Strategy: cada hechizo encapsula su propio algoritmo y todos son intercambiables porque comparten el mismo contrato.

```
public interface Hechizo {
    void ejecutar(Personaje lanzador, Personaje objetivo);
    TipoMagia getTipoMagia();
    int getNivelReq();
    String getNombre();
    boolean esTargetAliado();
    boolean haceDanio(); //si aparte de tener como objetivo un enemigo hace danio o solo le aplica un estado
}
```

Fragmento 3. El contrato común que cumplen los 14 hechizos.

La gran ventaja de esto es que agregar un hechizo nuevo no toca absolutamente nada de lo existente: se crea una clase, se implementa el efecto en su ejecutar, se la suma a la fábrica y listo. Implementamos catorce conjuros, repartidos en tres tipos según lo que hacen:

Hechizo	Tipo	Nivel	Objetivo	Efecto [Duración turnos]
Expelliarmus	Neutral	10	Enemigo	Daño base 20
Flipendo	Neutral	30	Enemigo	Daño base 30
Diffindo	Neutral	50	Enemigo	Daño base 40
Reducto	Luminosa	85	Enemigo	Daño 50 + 40% de confundir
Sectusempra	Luminosa	120	Enemigo	Daño base 70
Crucio	Oscura	95	Enemigo	Daño 70 · 40% falla · 30% confunde
Avada Kedavra	Oscura	140	Enemigo	Letal (×2 vida), pero falla el 80%
Episkey	Neutral	15	Aliado	Cura base 20
Ferula	Neutral	35	Aliado	Cura base 35
Vulnera Sanentur	Neutral	100	Aliado	Cura base 60
Protego	Neutral	50	Aliado	Estado PROTEGIDO [1] (mitiga el próximo golpe)
Petrificus Totalus	Luminosa	70	Enemigo	Estado PARALIZADO [1]
Imperio	Oscura	120	Enemigo	PARALIZADO [1] + CONFUNDIDO [2]
Reparifors	Neutral	75	Aliado	Limpia CONFUNDIDO y PARALIZADO

Tabla 3. Catálogo completo de hechizos implementados.

El TipoMagia (oscura, luminosa o neutral) es lo que define quién puede aprender qué. Los magos no tocan magia oscura y los mortífagos no acceden a la luminosa, los neutrales los puede usar cualquiera. Esa regla la resuelve cada bando en su puedeAprender:

```
//En Mago
@Override
public boolean puedeAprender(Hechizo h) {
    return h.getTipoMagia() != TipoMagia.OSCURA
    && this.getNivelMagia() >= h.getNivelReq();
}
```

Fragmento 4. La restricción de aprendizaje también es polimórfica: Mortifago invierte la regla.

El método `esTargetAliado` logra que en vez de que el batallón tenga que adivinar si un hechizo va a un amigo o a un enemigo, es el propio hechizo el que lo declara. `Protego`, `Episkey` o `Vulnera Sanentur` devuelven `true` y se lanzan sobre un compañero del mismo bando, los de ataque devuelven `false` y apuntan al enemigo. De nuevo, la información vive donde corresponde.

6. Creación centralizada: el patrón Factory

La consigna pedía ocultar los detalles de construcción, de modo que pedir "un mago" o "un hechizo" no obligue a saber cómo se arma por dentro. Para eso usamos dos fábricas.

La `HechizoFactory` centraliza la creación de conjuros. Tiene un método que devuelve un hechizo a partir de su nombre y otro, `obtenerTodos()`, que entrega la lista completa de los catorce. Este segundo método es el que después usa la fábrica de personajes.

La `PersonajeFactory` es la que más trabajo se ahorra. Cada método (`crearAuror`, `crearEstudiante`...) crea el personaje con un nivel de magia aleatorio dentro de un rango propio de su tipo y le asigna automáticamente todos los hechizos que es capaz de aprender. Y eso último lo resuelve apoyándose en el polimorfismo, sin saber de qué clase concreta se trata:

```
private static void asignarHechizos(Personaje p) {
    for (Hechizo h : HechizoFactory.obtenerTodos()) {
        if (p.puedeAprender(h)) {
            p.aprenderHechizo(h);
        }
    }
}
```

Fragmento 5. La fábrica de personajes reparte hechizos sin preguntar el tipo: cada uno filtra los suyos.

El detalle de los rangos de nivel le da personalidad a cada tipo sin esfuerzo extra. Un estudiante arranca entre 15 y 60 de nivel (apenas accede a hechizos básicos), un profesor entre 50 y 110, y un auror entre 100 y 150, lo que le permite aprender hasta los conjuros más exigentes. Del lado oscuro, el comandante (80-150) suele desbloquear el Imperio e incluso el `Avada Kedavra`, mientras que el seguidor común se queda en un rango más modesto. Como el nivel es aleatorio, no hay dos batallas exactamente iguales.

7. Gestión de batallones y colecciones

La clase `Batallon` es donde la consigna nos pedía usar distintos tipos de colección, cada una elegida por sus propiedades y no por costumbre. Terminamos usando las tres que sugería el enunciado, y cada una cumple un rol bien diferenciado:

Colección	Atributo	Por qué esa y no otra
List	integrantes	El orden importa: los personajes actúan en secuencia dentro del turno. Una lista mantiene ese orden y permite recorrerla.
Set	hechizosLanzadosEnTurno	Garantiza que un batallón no repita un hechizo en el mismo turno. El conjunto no

		admite duplicados, así que la regla se cumple sola.
Map	historialHechizosPorPersonaje	Asocia cada personaje con la lista de hechizos que fue lanzando. Es un registro clave -> valor, ideal para las estadísticas finales.

Tabla 4. Las tres colecciones del batallón y el criterio para elegir cada una.

El método central es `ejecutarTurno`. Al empezar el turno limpia el Set de hechizos usados y después recorre a los integrantes en pie. Por cada uno arma la lista de hechizos todavía disponibles (los que aún no se repitieron en ese turno), elige uno al azar, busca un objetivo (aliado o enemigo según lo que el hechizo declare) y lo ejecuta, registrando todo en el historial. Los personajes eliminados o paralizados se saltan, como manda la consigna.

Que la elección de hechizo y objetivo sea aleatoria le da variedad a cada partida y, de paso, hace que las pruebas tengan que contemplar el azar en vez de asumir un resultado fijo.

8. Reglas de la batalla

El programa principal, `BatallaMagosVsMortifagos`, arma los dos batallones con la fábrica y los hace combatir. La dinámica sigue lo que pedía el enunciado:

- **Se juega por rondas.** Antes de cada una se muestra el estado de vida de ambos bandos.
- **La iniciativa es al azar.** En cada ronda se sortea qué batallón ataca primero, el otro contraataca solo si todavía le queda gente en pie.
- **Cada integrante en pie actúa.** En su turno, el batallón hace que cada personaje vivo lance un hechizo (de ataque, de defensa o de curación).
- **Caer es definitivo.** Cuando la vida llega a cero, el personaje queda eliminado: no ataca ni puede ser elegido como objetivo.
- **La batalla termina** cuando un bando se queda sin personajes saludables. Ahí se anuncia al ganador y se muestran las estadísticas: vida restante e historial de hechizos de cada uno.

Le agregamos un par de cosas para que la demo sea más vistosa en la defensa: la consola se limpia entre rondas, hay una pausa interactiva (se avanza con Enter) y al final se imprime un resumen con el historial que veníamos guardando en el Map.

9. Bonus extra 1: Hechizos con efectos de estado temporales

Lo resolvimos con un enum `Estado` y un `Map<Estado, Integer>` dentro de cada personaje. Usar un Map nos permite que un personaje pueda acumular varios estados a la vez junto a la cantidad de turnos restantes que permanece. Actualmente hay tres estados y conviven sin problema:

Estado	Qué provoca	Cómo se aplica / se quita
PROTEGIDO	Reduce a la mitad el próximo golpe recibido y se consume al usarse.	Lo da Protego / se gasta en el primer impacto o hasta que se gaste por turnos.
PARALIZADO	El personaje pierde su acción en el turno.	Lo dan Petrificus Totalus e Imperio / lo limpian Reparifors o hasta que se gaste por

		turnos.
CONFUNDIDO	El siguiente hechizo de ataque le sale por la culata: se daña a sí mismo.	Lo dan Crucio, Reducto e Imperio /lo puede limpiar Reparifors o hasta que se gaste por turnos

Tabla 5. Estados disponibles y su efecto en combate.

La mejor parte del diseño es lo desacoplado que quedó. Los estados no están cableados dentro de los hechizos: el conjuro simplemente agrega o quita un estado, y es la mecánica del personaje la que decide qué significa tenerlo. Recibir daño, por ejemplo, consulta si hay un Protego activo y, si lo hay, parte el daño y lo consume:

```
public void recibirDanio(int danio) {
    danio = modificadorDefensa(danio);

    //Si la defensa redujo el daño a menos de 0, lo dejamos en 0 para que no cure
    if (danio < 0) {
        danio = 0;
    }

    //Si tiene Protego, se reduce a la mitad el daño restante
    if (this.tieneEstado(Estado.PROTEGIDO)) {
        danio /= 2;
        this.quitarEstado(Estado.PROTEGIDO);
    }

    //Restamos la vida final
    this.puntosVida -= danio;
    if (this.puntosVida < 0) {
        this.puntosVida = 0;
    }
}
```

Fragmento 6. La defensa y el estado PROTEGIDO se resuelven en un solo lugar, no en cada hechizo.

Gracias a esto, sumar un estado nuevo (un sangrado, una regeneración por turnos, lo que sea) no obliga a tocar los estados que ya están: alcanza con agregar el valor al enum y enseñarle al personaje a interpretarlo.

10. Bonus extra 2: Comandante reacciona a muerte de aliado

Además del bonus de los estados temporales aplicados a los distintos miembros del batallón, decidimos también agregarle un aumento de daño a la clase “Comandante” cuando un aliado suyo muere en batalla.

Esto lo logramos creando un nuevo estado:

Estado	Qué provoca	Cómo se aplica / se quita
VENGANZA	Se le aplica a un Comandante cuando un aliado suyo muere, otorgándole un aumento en el daño que va a realizar su	Se aplica con el fallecimiento de un miembro del equipo del Comandante, y se quita cuando el comandante realiza el

	siguiente ataque. (1.5%)	ataque potenciado
--	--------------------------	-------------------

Y, además, realizamos diversas modificaciones en distintas clases, en la clase Batallón, hay una función llamada “notificarCaidaDeAliado”, la cual se encarga de informarle al batallón del comandante que su aliado cayo (haciendo que entre, como consecuencia, en estado de venganza). Luego, se le aumenta el daño al Comandante a través del método “modificadorDanio” y se notifica a través del método “notificarAntesDeAtaqueDanio”, los cuales están dentro de la clase “Comandante”:

```
@Override
public void notificarAntesDeAtaqueDanio(Hechizo hechizo) {
    if (hechizo.haceDanio() && this.tieneEstado(Estado.VENGANZA)) {
        System.out.println(" >>> " + this.getNombre() + " canaliza su
            VENGANZA en el ataque (+50% de daño)!");
    }
}

@Override
public int modificadorDanio(int base) {
    int danio = super.modificadorDanio(base) + 15;
    if (this.tieneEstado(Estado.VENGANZA)) { //le aumento el danio si tenia
        //venganza activada
        danio = (int) (danio * MULTIPLICADOR_VENGANZA);
    }
    return danio;
}
```

Una vez realizado el ataque con el efecto de Venganza, se llama al método “despuesDeLanzarHechizo”, de la clase “Comandante”, el cual se encarga de quitarle el estado “VENGANZA” a ese Comandante (si lo tenía activado):

```
@Override
public void despuesDeLanzarHechizo(Hechizo hechizo) {
    if (hechizo.haceDanio() && this.tieneEstado(Estado.VENGANZA) &&
        !this.tieneEstado(Estado.CONFUNDIDO)) {
        this.quitarEstado(Estado.VENGANZA);
        System.out.println(" >>> " + this.getNombre() + " descarga su
            VENGANZA y vuelve a la normalidad.");
    }
}
```

Gracias a todo esto, tenemos en el batallón de los Mortifagos un Comandante el cual reacciona en tiempo real durante la batalla al fallecimiento de sus compañeros y actúa en consecuencia, potenciando su siguiente ataque gracias al estado “Venganza” y retirándoselo una vez que su ataque haya concluido.

11. Pruebas unitarias con JUnit

Escribimos alrededor de 64 pruebas repartidas en tres clases, una por área del sistema. La idea fue cubrir tanto los casos felices como los bordes molestos: que la vida no baje de cero, que no se aprendan hechizos prohibidos, que las colecciones se comporten, que la batalla siempre termine.

Clase de prueba	Casos	Qué valida
PersonajesTest	≈ 28	Vida y curación con tope, estados (Protego, parálisis), aprendizaje según tipo y nivel, y los modificadores polimórficos.
hechizosTest	≈ 20	La fábrica, que cada hechizo de daño reste vida, que los de curación la recuperen, los de estado y los tipos de magia.
CombateTest	≈ 16	Gestión del batallón, las colecciones de vivos, la lógica de turno y que la batalla termine con un único ganador.

Tabla 6. Distribución de las pruebas unitarias.

Un caso que nos hizo pensar fue el de los hechizos con azar. Crucio falla el 40% de las veces, así que no podíamos lanzarlo una vez y dar por hecho que iba a pegar. La solución fue lanzarlo en un bucle hasta verificar que el daño efectivamente se aplicó, y dejar el test como válido aun en el caso improbable de que fallara veinte veces seguidas. También verificamos los límites: que curar a alguien sano no lo lleve por encima de su vida máxima, que un personaje muerto o paralizado no pueda actuar, y que pasar un hechizo nulo lance la excepción correspondiente.

Para evitar que un test de batalla completa quedara colgado en un bucle infinito (algo posible si, por mala suerte, ambos bandos se la pasaran curándose), les pusimos un tope de 500 rondas y una aserción que confirma que el combate termina mucho antes.

12. Decisiones de diseño y dificultades

Varias cosas las ajustamos sobre la marcha y vale la pena dejarlas anotadas:

- **Balancear la pelea.** Las primeras versiones terminaban en dos turnos o se eternizaban. Tuvimos que afinar los modificadores, los rangos de nivel y las probabilidades de los conjuros más fuertes hasta que las batallas quedaran parejas y entretenidas.
- **El Avada Kedavra.** Es un hechizo que mata de un golpe, así que si pegaba siempre rompía el juego. Le pusimos un 80% de fallo: cuando entra, define la pelea, cuando no, abre una ventana para el rival.
- **Tope de vida.** Al principio la curación no tenía techo y los personajes llegaban a vidas absurdas. Agregamos un `puntosVidaMax` para que nadie supere su vida inicial.
- **Confusión "boomerang".** En vez de que CONFUNDIDO solo bajara la puntería, decidimos que el atacante confundido se pegue a sí mismo. Quedó más vistoso en la salida por consola y le da sentido a curar estados con `Reparifors`.
- **Hechizos de estado y daño.** Al principio los hechizos estaban separados entre hechizos que afectaban a aliados y hechizos que afectaban a enemigos, decidimos agregarles el método `"haceDanio"` para que además de saber si un hechizo afecta a un enemigo, le hace daño o solo le aplica estados negativos. (De esta forma solucionamos el bug de que venganza se consuma con un ataque que no hacia daño)

Si tuviéramos que seguir extendiéndolo, el diseño ya está preparado: sumar un personaje, un hechizo o un estado nuevo no implica reescribir lo que funciona, que era exactamente el objetivo de fondo del trabajo.

13. Conclusiones

El trabajo nos sirvió para ver, en un ejemplo concreto, por qué tanto se insiste con la herencia, el polimorfismo y los patrones de diseño. La consigna ponía una traba a propósito (prohibir los condicionales por tipo) y esa restricción, lejos de complicarnos, nos empujó hacia un diseño más limpio. Cuando cada clase sabe resolver lo suyo, el código que las usa se vuelve corto y deja de llenarse de casos especiales.

Las tres colecciones nos dejaron claro que elegir la estructura adecuada no es un detalle: el Set resolvió la regla de "no repetir hechizo" prácticamente solo, y el Map nos regaló las estadísticas finales sin esfuerzo. Las fábricas, por su lado, mantuvieron el resto del sistema ajeno a los detalles de construcción, que es justo lo que uno quiere cuando el proyecto empieza a crecer.

Pero quizás lo más valioso fueron las pruebas. Tener más de sesenta tests nos dio la confianza para cambiar cosas (rebalancear daños, agregar estados) sabiendo enseguida si habíamos roto algo. Terminamos con un sistema que cumple todos los puntos de la consigna, implementa dos de los bonus y, sobre todo, quedó fácil de extender. Si el día de mañana hay que sumar un nuevo tipo de mago o un hechizo inédito, sabemos exactamente dónde tocar y qué no hace falta tocar.

14. Referencias

HarryPotter Wiki. (s. f.). HarryPotter Wiki. Fandom. (Utilizada para elegir los nombres de Hechizos)

https://harrypotter.fandom.com/es/wiki/HarryPotter_Wiki

Refactoring. Guru. (s. f.). *Comparación entre Factory Method y Abstract Factory*. (Utilizada para crear Simple Factory en HechizosFactory)

<https://refactoring.guru/es/design-patterns/factory-comparison>